

Exercise 2

Advanced Methods for Regression and Classification

Kyzer Gerez

23.10.2025

```
load("building.RData")

# split into 2/3 train 1/3 test
smp_size <- floor(2/3 * nrow(df))

# set the seed to make partition reproducible
set.seed(1)
train_ind <- sample(seq_len(nrow(df)), size = smp_size)

train <- df[train_ind, ]
test <- df[-train_ind, ]
```

- (1) Compute the full model with `lm()` for the training data, and predict the response values for the test data.

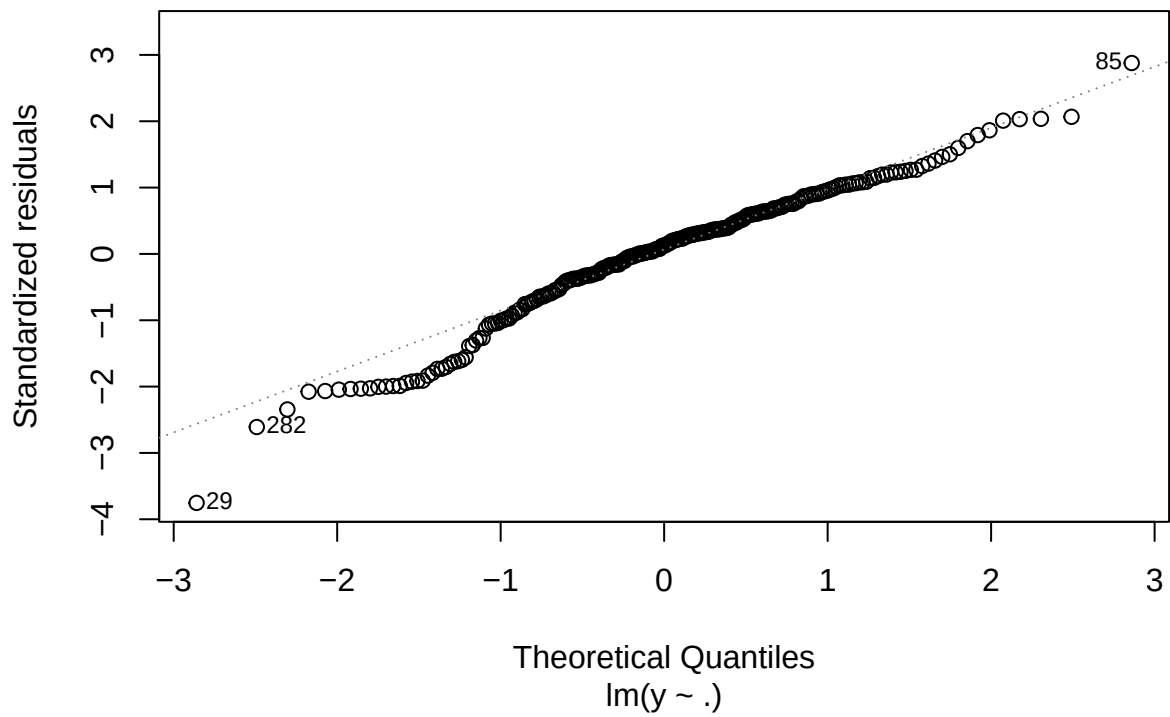
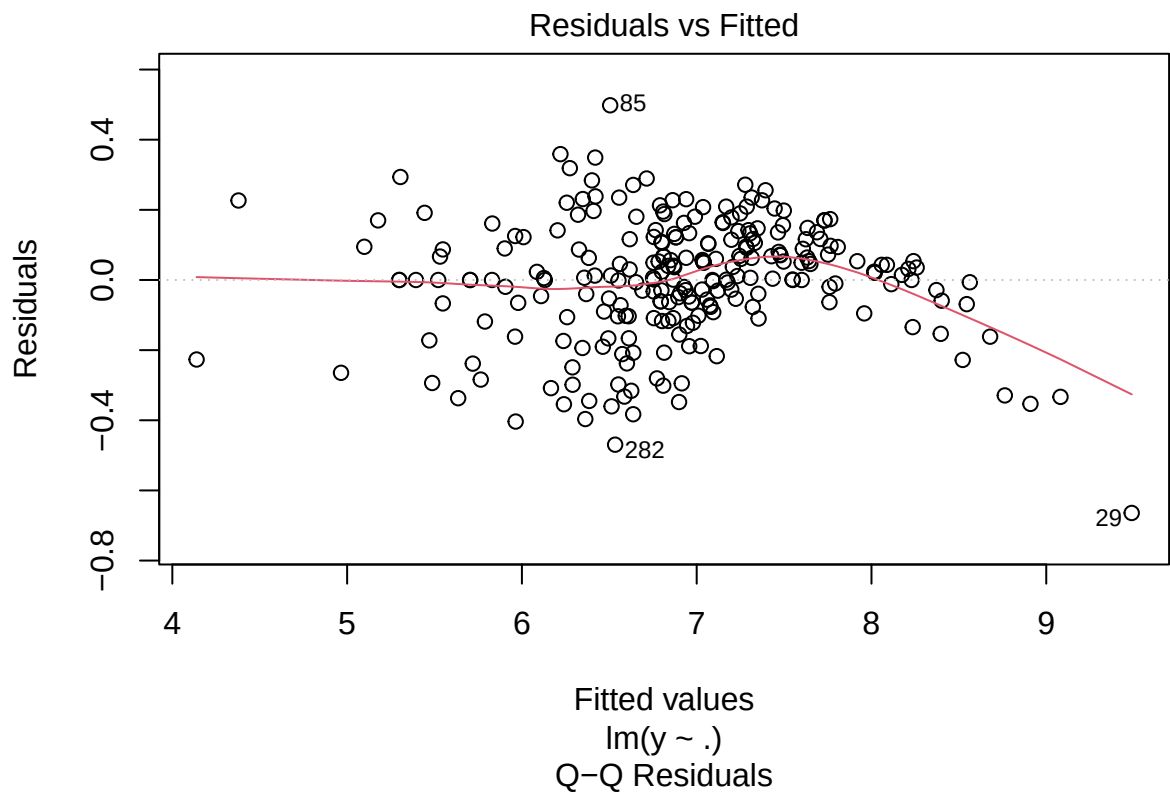
```
full_model <- lm(y ~ ., train)
test_pred <- predict(full_model, test)
```

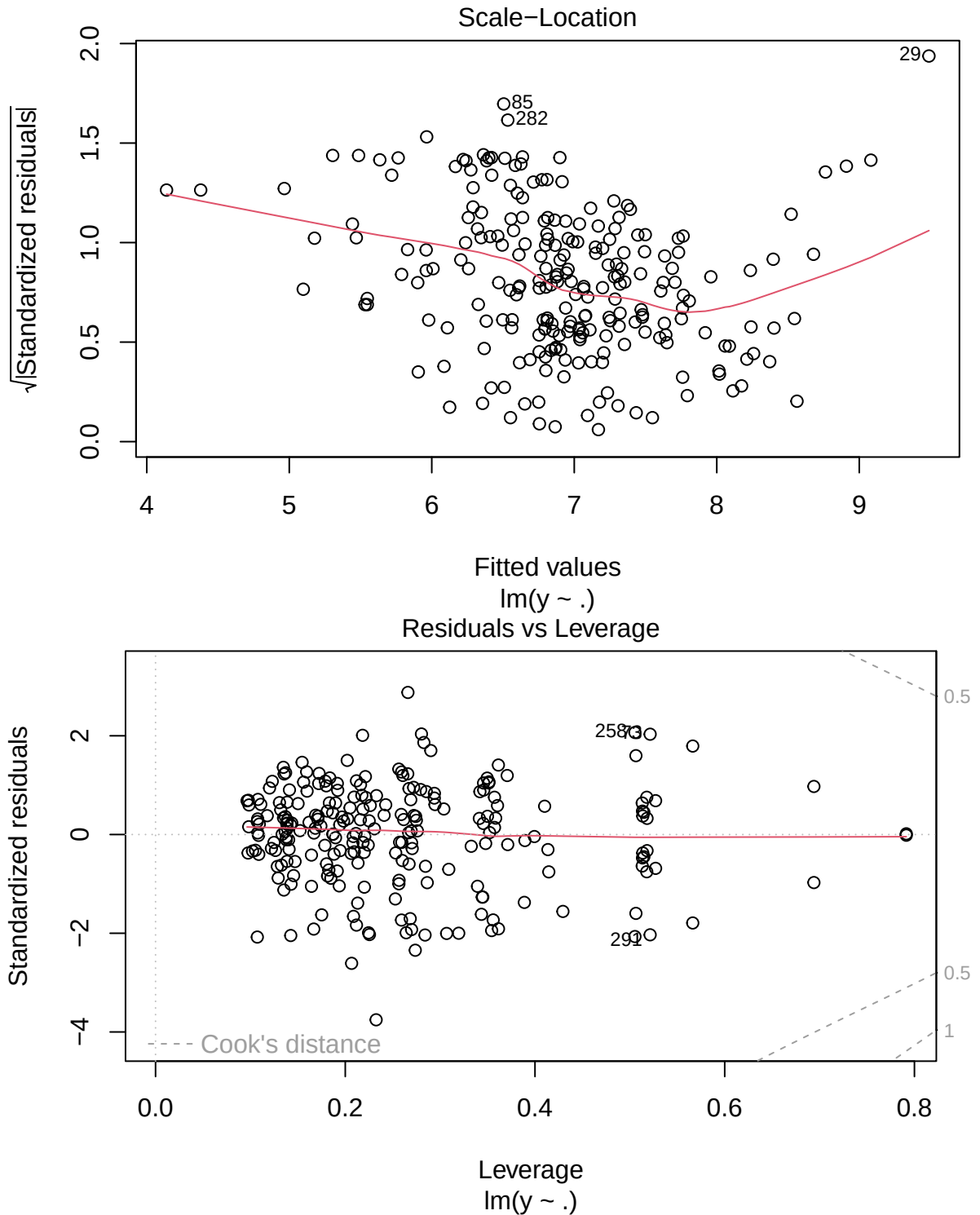
```
## Warning in predict.lm(full_model, test): prediction from rank-deficient fit;
## attr(*, "non-estim") has doubtful cases
```

- (a) Use `plot()` for the result object and briefly discuss the findings.

```
plot(full_model)
```

```
## Warning: not plotting observations with leverage one:
## 30, 38, 44, 58, 93, 98, 120, 166, 203, 204, 206, 221
```





The residuals vs fitted plot indicates some nonlinearity when looking at and above the fitted value 7. The Q-Q plot suggests that the residuals are normally distributed, with the only outlier being observation 29, which was also noted in the previous plot. The scale-location plot shows that the assumption of equal variance may not hold, with a wider spread along the middle of the x-axis range. No datapoints stand out in the residuals vs leverage plot. This indicates that datapoint 29 was not influential despite being notable in the other plots.

- (b) Look at the estimated regression coefficients. Depending on your training/test split these might include NA's. What could be the reason for that?

```
summary(full_model)
```

```
##
## Call:
## lm(formula = y ~ ., data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.66423 -0.08015  0.00884  0.11627  0.49808
##
## Coefficients: (35 not defined because of singularities)
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   -2.105e+01  2.717e+02  -0.077  0.938334
## START.YEAR      3.330e-01  3.693e+00   0.090  0.928272
## START.QUARTER  -1.535e+00  8.962e-01  -1.713  0.088515 .
## COMPLETION.YEAR  9.923e-02  3.553e-02   2.793  0.005803 **
## COMPLETION.QUARTER 2.144e-02  1.667e-02   1.287  0.199901
## PhysFin1       -2.332e-02  4.894e-03  -4.766  3.94e-06 ***
## PhysFin2        1.262e-04  3.933e-05   3.209  0.001587 **
## PhysFin3       -1.726e-04  1.111e-04  -1.553  0.122207
## PhysFin4       -4.199e-04  1.107e-04  -3.794  0.000204 ***
## PhysFin5       -1.845e-03  6.729e-04  -2.742  0.006734 **
## PhysFin6        6.519e-04  1.790e-04   3.642  0.000357 ***
## PhysFin7                NA         NA      NA      NA
## PhysFin8        5.366e-04  3.237e-05  16.576 < 2e-16 ***
## Econ1          1.207e-04  4.632e-04   0.261  0.794765
## Econ2         -7.287e-01  9.898e-01  -0.736  0.462581
## Econ3          3.339e-01  3.225e-01   1.035  0.301930
## Econ4          2.977e-01  2.798e-01   1.064  0.288932
## Econ5         -1.022e-04  5.631e-05  -1.815  0.071222 .
## Econ6         -2.065e-03  8.964e-04  -2.303  0.022432 *
## Econ7         -1.344e-01  2.138e-01  -0.629  0.530280
## Econ8          1.067e-02  1.887e-02   0.565  0.572462
## Econ9         -1.722e-04  1.349e-04  -1.277  0.203361
## Econ10         7.796e-01  3.821e-01   2.041  0.042798 *
## Econ11        -2.301e-03  2.406e-03  -0.956  0.340197
## Econ12         4.981e-03  2.833e-03   1.758  0.080472 .
## Econ13        -4.097e-04  1.482e-04  -2.764  0.006321 **
## Econ14         3.924e-04  1.425e-03   0.275  0.783428
## Econ15        -3.776e-01  2.900e-01  -1.302  0.194672
## Econ16         9.798e-01  3.586e-01   2.733  0.006928 **
## Econ17         1.795e-04  6.959e-04   0.258  0.796763
## Econ18        -9.616e-05  1.004e-04  -0.957  0.339700
## Econ19        -2.070e-05  9.116e-06  -2.271  0.024384 *
## Econ1.lag1     -1.076e-03  4.173e-04  -2.577  0.010777 *
## Econ2.lag1     -1.086e+00  1.005e+00  -1.081  0.281319
## Econ3.lag1     -4.404e-02  1.777e-01  -0.248  0.804538
## Econ4.lag1      2.294e+00  1.115e+00   2.058  0.041121 *
## Econ5.lag1      1.654e-05  4.893e-05   0.338  0.735710
## Econ6.lag1     -7.770e-04  1.164e-03  -0.668  0.505295
## Econ7.lag1      3.328e-02  2.575e-01   0.129  0.897311
## Econ8.lag1      2.789e-03  5.571e-03   0.501  0.617261
```

## Econ9.lag1	2.903e-05	8.431e-05	0.344	0.730998	
## Econ10.lag1	-7.735e-01	3.373e-01	-2.293	0.023035	*
## Econ11.lag1	2.775e-03	4.329e-03	0.641	0.522314	
## Econ12.lag1	6.633e-04	1.817e-03	0.365	0.715565	
## Econ13.lag1	7.043e-04	6.978e-04	1.009	0.314186	
## Econ14.lag1	2.409e-04	1.639e-03	0.147	0.883273	
## Econ15.lag1	1.197e+00	5.703e-01	2.098	0.037313	*
## Econ16.lag1	-4.604e-01	4.187e-01	-1.100	0.272949	
## Econ17.lag1	-1.444e-03	7.179e-04	-2.012	0.045746	*
## Econ18.lag1	-2.172e-04	1.090e-04	-1.993	0.047845	*
## Econ19.lag1	-3.191e-06	1.577e-05	-0.202	0.839900	
## Econ1.lag2	-1.661e-04	2.973e-04	-0.559	0.576941	
## Econ2.lag2	1.638e+00	5.048e-01	3.244	0.001413	**
## Econ3.lag2	1.154e-01	4.815e-01	0.240	0.810847	
## Econ4.lag2	-6.864e-01	4.933e-01	-1.391	0.165876	
## Econ5.lag2	2.009e-04	8.271e-05	2.428	0.016174	*
## Econ6.lag2	8.844e-05	1.115e-03	0.079	0.936838	
## Econ7.lag2	-5.075e-02	1.500e-01	-0.338	0.735498	
## Econ8.lag2	-6.014e-03	1.338e-02	-0.450	0.653535	
## Econ9.lag2	-1.184e-04	1.117e-04	-1.060	0.290412	
## Econ10.lag2	6.562e-01	5.264e-01	1.246	0.214275	
## Econ11.lag2	1.734e-03	3.557e-03	0.487	0.626529	
## Econ12.lag2	3.635e-03	4.735e-03	0.768	0.443671	
## Econ13.lag2	4.449e-04	8.564e-04	0.519	0.604089	
## Econ14.lag2	-5.086e-04	4.804e-04	-1.059	0.291187	
## Econ15.lag2	-6.990e-01	5.619e-01	-1.244	0.215180	
## Econ16.lag2	-6.645e-01	5.291e-01	-1.256	0.210833	
## Econ17.lag2	9.711e-04	6.377e-04	1.523	0.129636	
## Econ18.lag2	-1.075e-04	1.019e-04	-1.055	0.293038	
## Econ19.lag2	3.893e-06	4.330e-06	0.899	0.369849	
## Econ1.lag3	2.940e-04	3.748e-04	0.784	0.433842	
## Econ2.lag3	-3.712e-01	3.666e-01	-1.013	0.312597	
## Econ3.lag3	-1.843e-01	7.626e-01	-0.242	0.809358	
## Econ4.lag3	4.648e-01	2.561e-01	1.815	0.071247	.
## Econ5.lag3	NA	NA	NA	NA	
## Econ6.lag3	NA	NA	NA	NA	
## Econ7.lag3	NA	NA	NA	NA	
## Econ8.lag3	NA	NA	NA	NA	
## Econ9.lag3	NA	NA	NA	NA	
## Econ10.lag3	NA	NA	NA	NA	
## Econ11.lag3	NA	NA	NA	NA	
## Econ12.lag3	NA	NA	NA	NA	
## Econ13.lag3	NA	NA	NA	NA	
## Econ14.lag3	NA	NA	NA	NA	
## Econ15.lag3	NA	NA	NA	NA	
## Econ16.lag3	NA	NA	NA	NA	
## Econ17.lag3	NA	NA	NA	NA	
## Econ18.lag3	NA	NA	NA	NA	
## Econ19.lag3	NA	NA	NA	NA	
## Econ1.lag4	NA	NA	NA	NA	
## Econ2.lag4	NA	NA	NA	NA	
## Econ3.lag4	NA	NA	NA	NA	
## Econ4.lag4	NA	NA	NA	NA	
## Econ5.lag4	NA	NA	NA	NA	

```
## Econ6.lag4          NA          NA          NA          NA
## Econ7.lag4          NA          NA          NA          NA
## Econ8.lag4          NA          NA          NA          NA
## Econ9.lag4          NA          NA          NA          NA
## Econ10.lag4         NA          NA          NA          NA
## Econ11.lag4         NA          NA          NA          NA
## Econ12.lag4         NA          NA          NA          NA
## Econ13.lag4         NA          NA          NA          NA
## Econ14.lag4         NA          NA          NA          NA
## Econ15.lag4         NA          NA          NA          NA
## Econ16.lag4         NA          NA          NA          NA
## Econ17.lag4         NA          NA          NA          NA
## Econ18.lag4         NA          NA          NA          NA
## Econ19.lag4         NA          NA          NA          NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.202 on 175 degrees of freedom
## Multiple R-squared:  0.9563, Adjusted R-squared:  0.9383
## F-statistic: 53.16 on 72 and 175 DF,  p-value: < 2.2e-16
```

35 predictors don't have defined coefficients because each of them have an exact linear relationship with at least one other predictor.

(c) Compute the RMSE (=square-root of the MSE) for training and test data, and discuss the findings.

```
library('Metrics')

train_pred <- predict(full_model, train)

precision <- 3
train_RMSE <- round(rmse(train$y, train_pred), precision)
test_RMSE <- round(rmse(test$y, test_pred), precision)

print(paste('Train RMSE:', train_RMSE))

## [1] "Train RMSE: 0.17"

print(paste('Test RMSE:', test_RMSE))

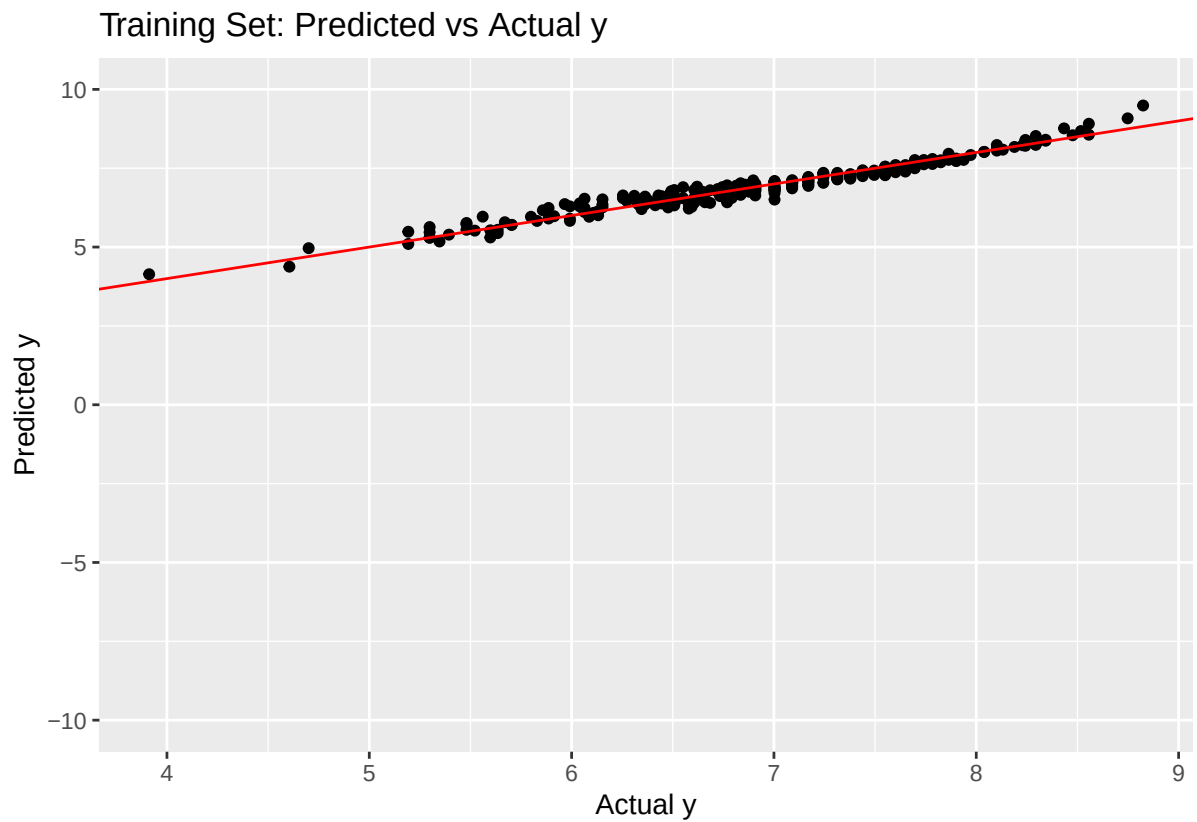
## [1] "Test RMSE: 1.508"
```

The test RMSE is significantly larger than the train RMSE, which suggests that the full model has an overfitting problem/high variance.

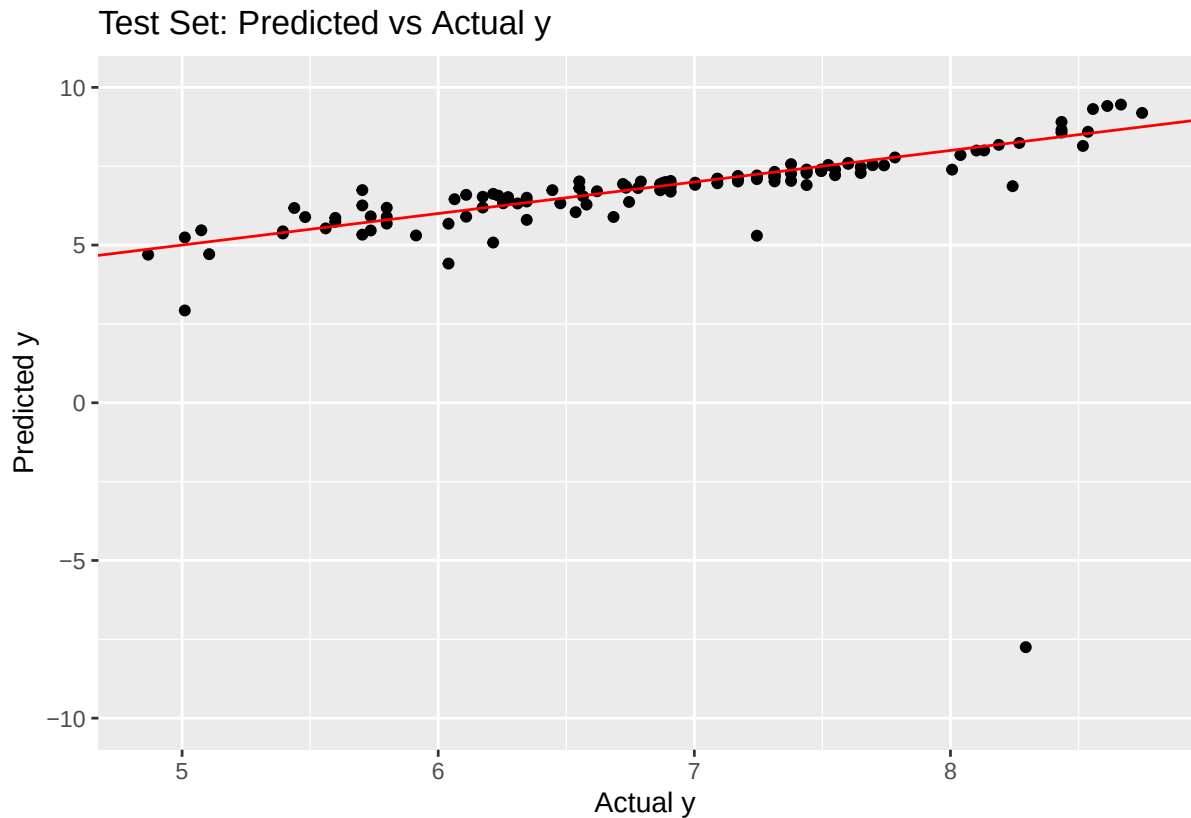
(d) Plot the observed response values versus the fitted/predicted values for training and test data, and discuss the findings.

```
library(ggplot2)

# train
ggplot(data.frame(actual = train$y, predicted = train_pred),
  aes(x = actual, y = predicted)) +
  ylim(-10, 10) +
  geom_point() +
  geom_abline(color = "red") +
  labs(title = "Training Set: Predicted vs Actual y",
    x = "Actual y", y = "Predicted y")
```



```
# test
ggplot(data.frame(actual = test$y, predicted = test_pred),
  aes(x = actual, y = predicted)) +
  ylim(-10, 10) +
  geom_point() +
  geom_abline(color = "red") +
  labs(title = "Test Set: Predicted vs Actual y",
    x = "Actual y", y = "Predicted y")
```



The training set plot shows that the predicted y is generally close to the actual y. At the higher end of actual y, the predicted y does tend to overestimate though.

The test set shows much larger residuals across the scale, with one data point having a predicted y that is far below the actual y.

- (2) Perform variable selection based on stepwise regression, using the function `step()`, see help file and course notes. Perform both, forward selection (start from the empty model) and backward selection (start from the full model). Compare the resulting models with the RMSE, and with plots of response versus predicted values.

```
forward_selection <- step(lm(y ~ 1, data = df),
  direction = "forward",
  scope = formula(full_model),
  trace=0)
backward_selection <- step(full_model,
  direction = "backward",
  trace=0)
```

```
# train - predict
forward_train_pred <- predict(forward_selection, train)
backward_train_pred <- predict(backward_selection, train)

# train - RMSE
forward_train_RMSE <- round(rmse(train$y, forward_train_pred), precision)
backward_train_RMSE <- round(rmse(train$y, backward_train_pred), precision)

# test - predict
```



```

forward_test_pred <- predict(forward_selection, test)
backward_test_pred <- predict(backward_selection, test)

# test - RMSE
forward_test_RMSE <- round(rmse(test$y, forward_test_pred), precision)
backward_test_RMSE <- round(rmse(test$y, backward_test_pred), precision)

# print
print(paste('Forward train RMSE:', forward_train_RMSE))

## [1] "Forward train RMSE: 0.209"
print(paste('Backward train RMSE:', backward_train_RMSE))

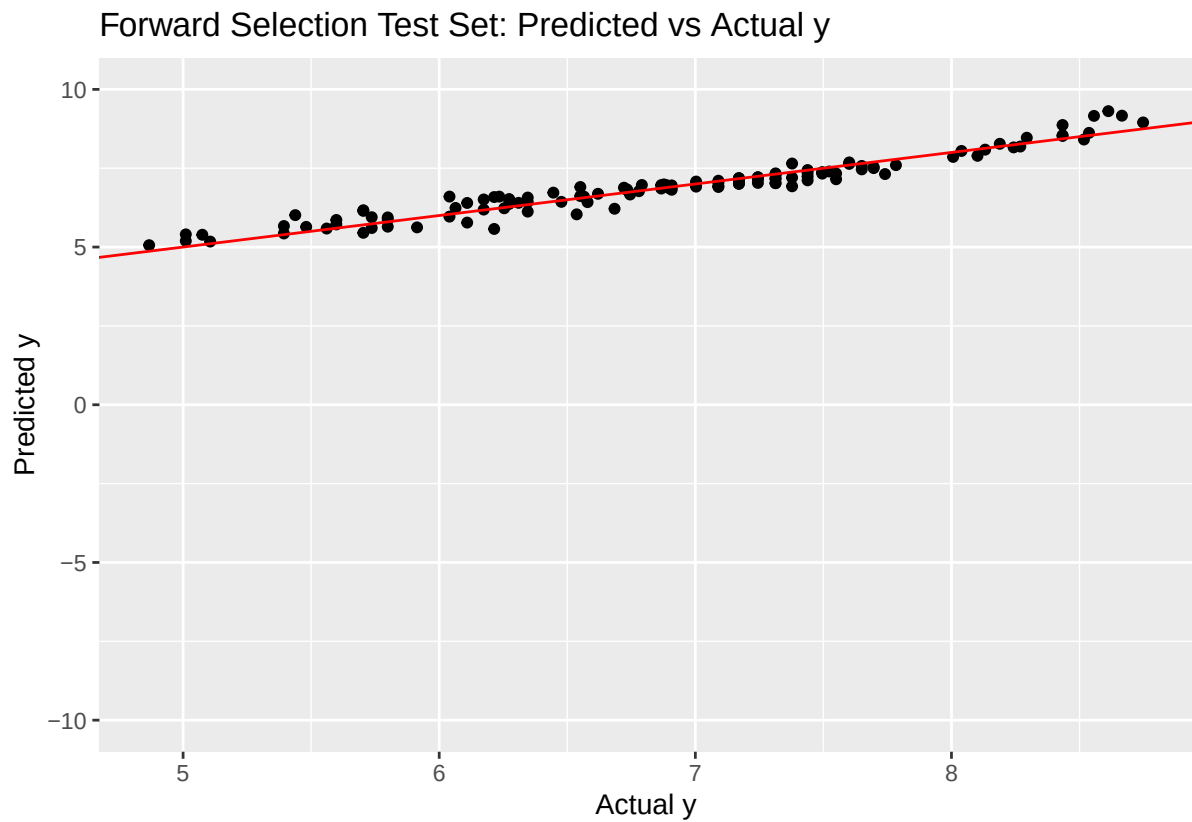
## [1] "Backward train RMSE: 0.172"
print(paste('Forward test RMSE:', forward_test_RMSE))

## [1] "Forward test RMSE: 0.236"
print(paste('Backward test RMSE:', backward_test_RMSE))

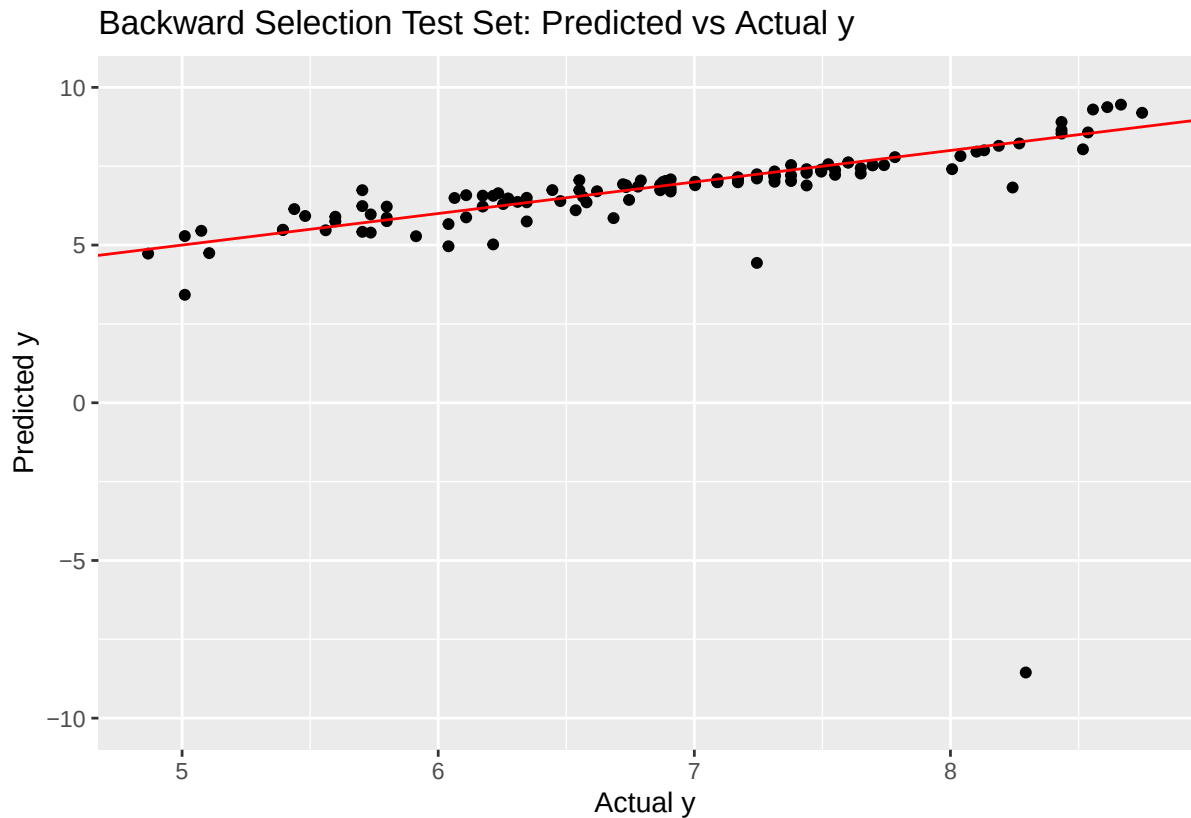
## [1] "Backward test RMSE: 1.58"

# plotting - test only
ggplot(data.frame(actual = test$y, predicted = forward_test_pred),
aes(x = actual, y = predicted)) +
ylim(-10, 10) +
geom_point() +
geom_abline(color = "red") +
labs(title = "Forward Selection Test Set: Predicted vs Actual y",
x = "Actual y", y = "Predicted y")

```



```
ggplot(data.frame(actual = test$y, predicted = backward_test_pred),  
  aes(x = actual, y = predicted)) +  
  ylim(-10, 10) +  
  geom_point() +  
  geom_abline(color = "red") +  
  labs(title = "Backward Selection Test Set: Predicted vs Actual y",  
    x = "Actual y", y = "Predicted y")
```



- (3) Which of the models from (1) and (2) would be the preferred one, and why? Would a model comparison based on ANOVA be useful?

Looking at the RMSE and the predicted vs actual y plots, the forward selection model greatly outperforms the full model and backward selection model. A model comparison based on ANOVA seems to be unnecessary since the simpler model based on forward selection is much better. ANOVA is useful when the more complex model outperforms and a test is needed to determine if this outperformance justifies the increased complexity.

- (4) Use the function `cvFit()` from the package `cvTools` to compare the models from (1) and (2), separately for training and test set observations (thus 6 results). A look at the help file shows how you can use the function for model evaluation based on cross-validation (CV). Set the parameters `cost=rmspe` for RMSE calculations, and perform 5-fold CV with 100 replications. Compare all 6 error distributions (available in `$reps`) by parallel boxplots. What do you conclude?

```
library(cvTools)

## Loading required package: lattice
## Loading required package: robustbase
##
## Attaching package: 'cvTools'
## The following object is masked from 'package:Metrics':
##
##     mape
# --- train ---
full_cv_RMSE_train <- cvFit(full_model, data=train, y=train$,
                             cost=rmspe, K=5, R=100, seed=1)
forward_cv_RMSE_train <- cvFit(forward_selection, data=train, y=train$,
```

```

        cost=rmspe, K=5, R=100, seed=1)
backward_cv_RMSE_train <- cvFit(backward_selection, data=train, y=train$,
                                cost=rmspe, K=5, R=100, seed=1)

# --- test ---
full_cv_RMSE_test <- cvFit(full_model, data=test, y=test$,
                           cost=rmspe, K=5, R=100, seed=1)
forward_cv_RMSE_test <- cvFit(forward_selection, data=test, y=test$,
                              cost=rmspe, K=5, R=100, seed=1)
backward_cv_RMSE_test <- cvFit(backward_selection, data=test, y=test$,
                               cost=rmspe, K=5, R=100, seed=1)

# create dfs for plotting
train_errors_df <- class(data.frame)
train_errors_df$rmse <- data.frame(
  Error = c(full_cv_RMSE_train$reps,
            forward_cv_RMSE_train$reps,
            backward_cv_RMSE_train$reps),
  Model = factor(rep(c("Full", "Forward", "Backward"),
                    times = c(length(full_cv_RMSE_train$reps),
                              length(forward_cv_RMSE_train$reps),
                              length(backward_cv_RMSE_train$reps)))))

## Warning in train_errors_df$rmse <- data.frame(Error = 
## c(full_cv_RMSE_train$reps, : Coercing LHS to a list

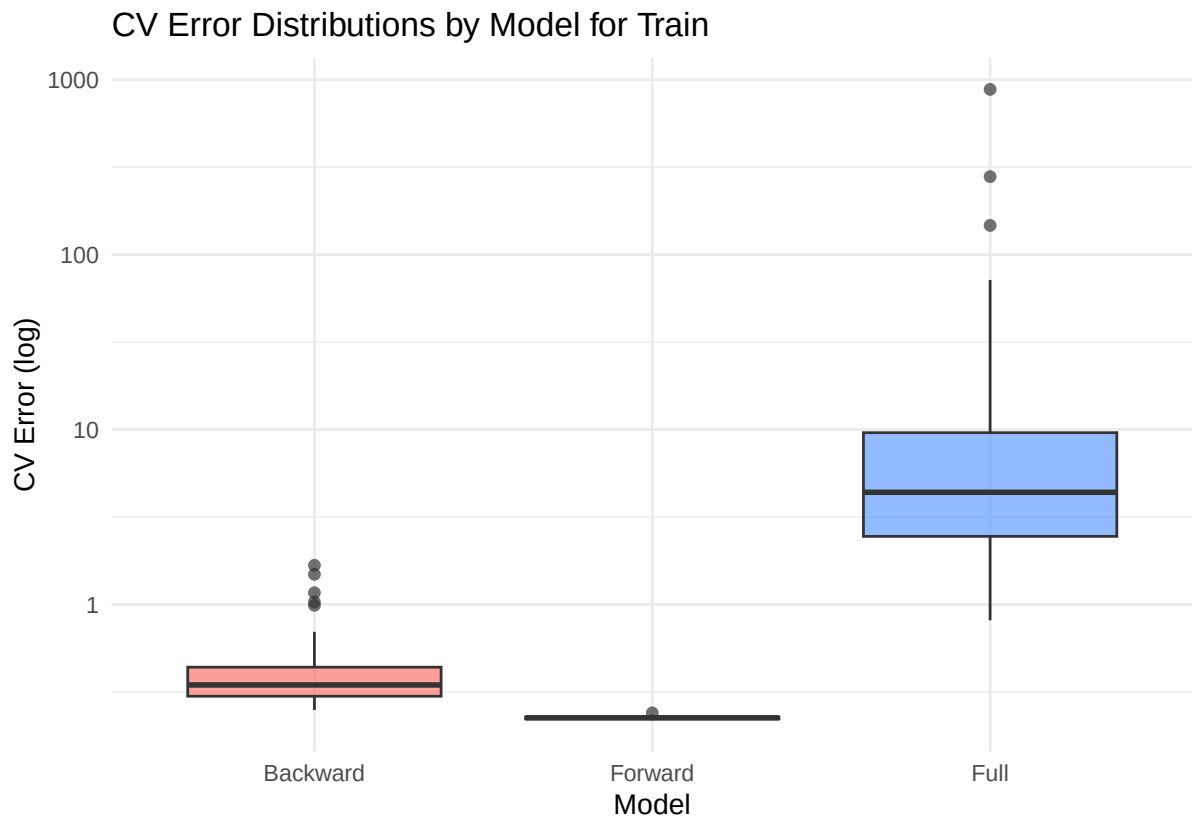
test_errors_df <- class(data.frame)
test_errors_df$rmse <- data.frame(
  Error = c(full_cv_RMSE_test$reps,
            forward_cv_RMSE_test$reps,
            backward_cv_RMSE_test$reps),
  Model = factor(rep(c("Full", "Forward", "Backward"),
                    times = c(length(full_cv_RMSE_test$reps),
                              length(forward_cv_RMSE_test$reps),
                              length(backward_cv_RMSE_test$reps)))))

## Warning in test_errors_df$rmse <- data.frame(Error = c(full_cv_RMSE_test$reps, 
## : Coercing LHS to a list

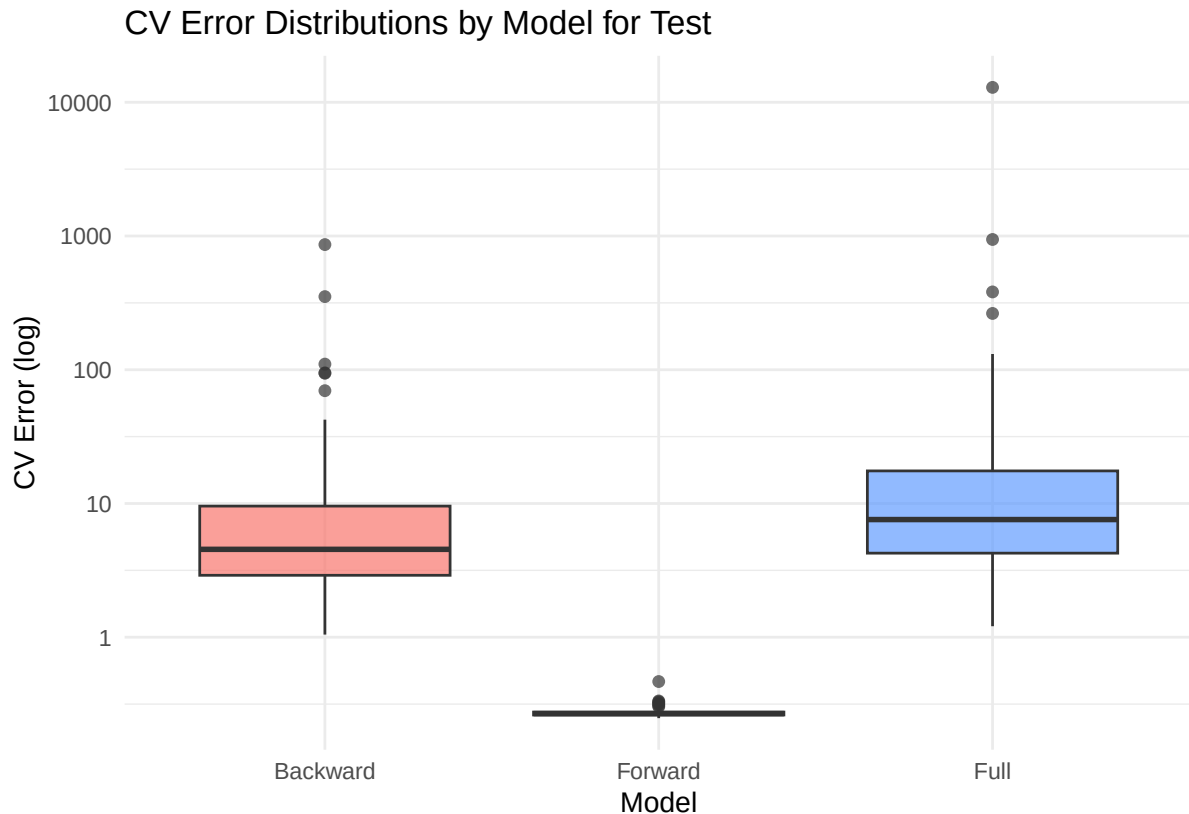
library(ggplot2)

ggplot(train_errors_df$rmse, aes(x = Model, y = Error, fill = Model)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 16, outlier.size = 2) +
  scale_y_continuous(trans='log10') +
  labs(
    title = "CV Error Distributions by Model for Train",
    y = "CV Error (log)",
    x = "Model"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

```



```
ggplot(test_errors_df$rmse, aes(x = Model, y = Error, fill = Model)) +  
  scale_y_continuous(trans='log10') +  
  geom_boxplot(alpha = 0.7, outlier.shape = 16, outlier.size = 2) +  
  labs(  
    title = "CV Error Distributions by Model for Test",  
    y = "CV Error (log)",  
    x = "Model"  
  ) +  
  theme_minimal() +  
  theme(legend.position = "none")
```



The full model is far more unstable due to significant outliers compared to the variable selection models. The forward model is much more stable over all replications and across different folds, and it has the lowest median error for both the train and test sets. The backward model is an improvement over the full model but still suffers from overfitting and sensitivity to outliers.

- (5) Do the same as in (4), but use `cost=rtmspe`. What are we doing? Which model is the preferred one?

```
# --- train ---
full_cv_RTMSPE_train <- cvFit(full_model, data=train, y=train$y,
                              cost=rtmspe, K=5, R=100, seed=1)
forward_cv_RTMSPE_train <- cvFit(forward_selection, data=train, y=train$y,
                                 cost=rtmspe, K=5, R=100, seed=1)
backward_cv_RTMSPE_train <- cvFit(backward_selection, data=train, y=train$y,
                                  cost=rtmspe, K=5, R=100, seed=1)

# --- test ---
full_cv_RTMSPE_test <- cvFit(full_model, data=test, y=test$y,
                              cost=rtmspe, K=5, R=100, seed=1)
forward_cv_RTMSPE_test <- cvFit(forward_selection, data=test, y=test$y,
                                 cost=rtmspe, K=5, R=100, seed=1)
backward_cv_RTMSPE_test <- cvFit(backward_selection, data=test, y=test$y,
                                  cost=rtmspe, K=5, R=100, seed=1)

# create dfs for plotting
train_errors_df$rtmse <- data.frame(
  Error = c(full_cv_RTMSPE_train$reps,
            forward_cv_RTMSPE_train$reps,
            backward_cv_RTMSPE_train$reps),
  Model = factor(rep(c("Full", "Forward", "Backward"),
```

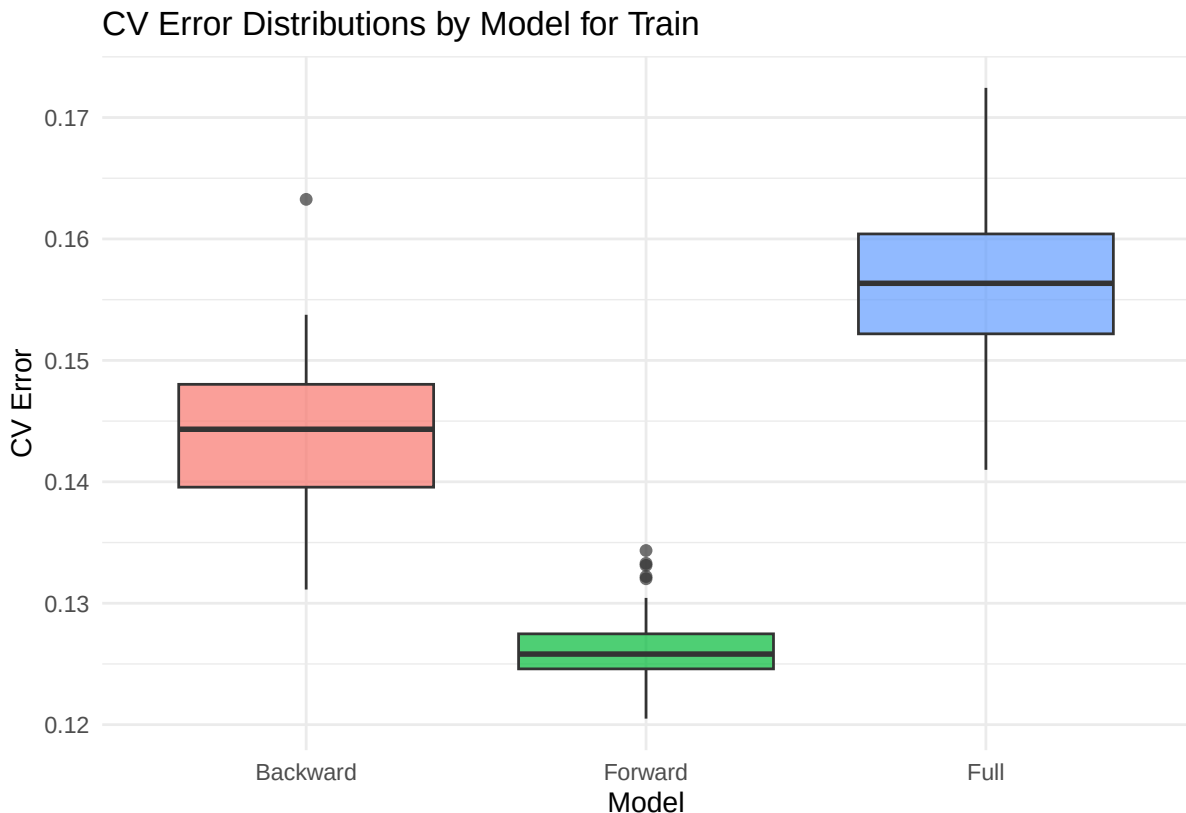
```

times = c(length(full_cv_RTMSE_train$reps),
           length(forward_cv_RTMSE_train$reps),
           length(backward_cv_RTMSE_train$reps)))

test_errors_df$rtmse <- data.frame(
  Error = c(full_cv_RTMSE_test$reps,
            forward_cv_RTMSE_test$reps,
            backward_cv_RTMSE_test$reps),
  Model = factor(rep(c("Full", "Forward", "Backward"),
                    times = c(length(full_cv_RTMSE_test$reps),
                              length(forward_cv_RTMSE_test$reps),
                              length(backward_cv_RTMSE_test$reps)))))

ggplot(train_errors_df$rtmse, aes(x = Model, y = Error, fill = Model)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 16, outlier.size = 2) +
  labs(
    title = "CV Error Distributions by Model for Train",
    y = "CV Error",
    x = "Model"
  ) +
  theme_minimal() +
  theme(legend.position = "none")

```



```

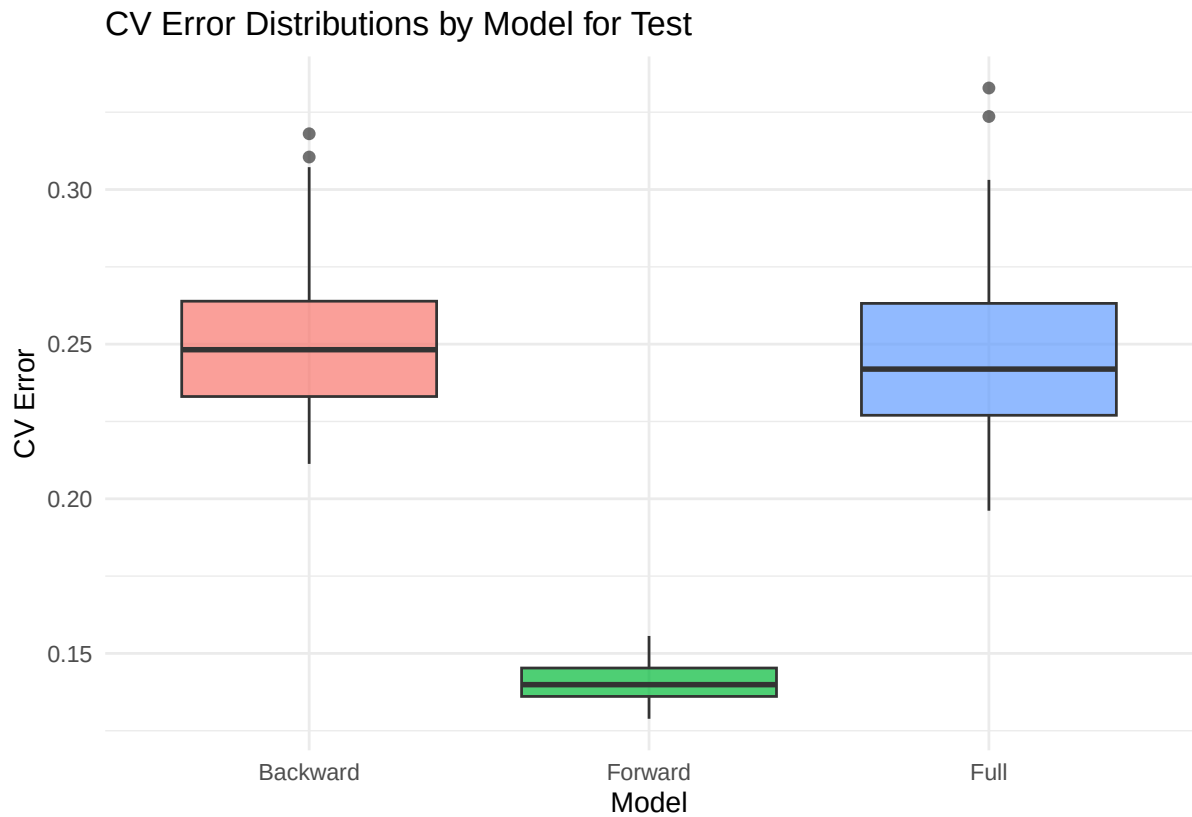
ggplot(test_errors_df$rtmse, aes(x = Model, y = Error, fill = Model)) +
  geom_boxplot(alpha = 0.7, outlier.shape = 16, outlier.size = 2) +
  labs(
    title = "CV Error Distributions by Model for Test",
    y = "CV Error",

```

```

x = "Model"
) +
theme_minimal() +
theme(legend.position = "none")

```



RTMSPE is a cost function that trims the largest squared differences of the observed and fitted values (25% by default). This means that outliers are excluded. Because of this, the backward and full models don't appear as unstable compared to using RMSPE. Still, the forward model is superior to the other two.